

COURSEWORK FEEDBACK: HURDLE

CS141

2127935

February 2022

This document lays out your scores and commentary feedback for the first coursework of CS141 Functional Programming.

Ex. 1	4/4	Good.
Ex. 2	2/4	You have a correct implementation that is explained well, although you could simplify it greatly by making use of <code>elem</code> instead from the standard prelude.
Ex. 3	4/6	Mostly good, although you could use a <code>let</code> binding or a <code>where</code> clause to avoid repeating <code>normalise</code> x three times.
Ex. 4	4/8	Your implementation of <code>exactMatches</code> is mostly correct, although it only works for words of equal length. It is still possible to have a matching between words of unequal length, in this case the matching has length equal to the shorter word. You could correct this by having two base cases (one for when only the first word is empty and another for when the only the second word is empty). Your comments are good at explaining how your code works, although you could improve these by considering whether a more elegant solution is possible; you could look into using <code>zip</code> or <code>zipWith</code> for example. Your code is clear to follow, although making use of <code>(++)</code> to prepend a value to a list is not very elegant; instead you should make use of <code>(:)</code> , e.g. <code>IsExact g : exactMatches gs as</code> .
Ex. 5	4/8	You have tried to write a correct definition of <code>removeExacts</code> , although again this function also only works when given two lists of equal length. You could again correct this by modifying the base case. Again you should avoid using <code>(++)</code> to prepend to a list and instead favour <code>(:)</code> . Using <code>(++)</code> with an empty list and a non-empty list can also just be simplified to the non-empty list, e.g. you can write <code>removeExacts es as</code> instead of <code>[] ++ removeExacts es as</code> .
Ex. 6	7/12	You have a correct implementation of <code>getMatches</code> , although the code is very complex. You have tried to explain how it works in your comments, although the guards used within <code>getMatches</code> are not very clear and would benefit from a further breakdown with regards to how these work. Again you should avoid using <code>(++)</code> to prepend values to lists and you could make use of <code>delete</code> from <code>Data.List</code> to avoid the need to write your own <code>removeItem</code> function.

COURSEWORK FEEDBACK: HURDLE

CS141

2127935

February 2022

Ex. 7	6/12	Your implemenation of <code>matchingAlgo</code> is correct. You have tried to explain how it works in your comments, but these could be improved by a more thorough explanation about what each of the subexpressions are doing within <code>matchingAlgo</code> . You could also use a <code>let</code> binding or a <code>where</code> clause to avoid the need to use <code>exactMatches</code> guess answer multiple times.
-------	------	---

Ex. 8	7/7	Good.
-------	-----	-------

Ex. 9	6/9	Your implemenation of <code>eliminateAll</code> is correct and explained well in your comments. Your code is clear to follow, though there are a few points where its elegance can be improved. As <code>apply</code> is only used as part of <code>eliminateAll</code> , this would be better written within a <code>let</code> binding or a <code>where</code> clause. You could also make use of a <code>fold</code> with <code>uncurry</code> from the standard prelude to avoid the need for explicit recursion and <code>fst</code> and <code>snd</code> .
-------	-----	--

Ex. 10					
Ingenuity	11/12	Code Quality	6/9	Explanation	7/9
You have considered many different approaches to improve the naive guessing algorithm and thoroughly explained your thought process for these approaches. It would be nice to see your final implementation using the more complex entropy approach that you considered, although you did justify against this as your entropy approach is rather slow. You have tried to make your attempts elegant, although you are using explicit recursion alot and a few of your attempts have many nested parentheses which are difficult to read, e.g. in <code>rankAlgo</code> and <code>p</code> . Splitting these up using <code>let</code> bindings or <code>where</code> clauses would be good. Making use of suitable abstractions and implicitly recursive functions from the standard prelude would be better than relying upon explicit recursion. Your comments would also benefit from an explanation of how <code>replicateM</code> works as you are using this for your entropy-based solution. An explanation of the List monad here would also be appropriate.					

General Comment	It is clear that you can solve problems in Haskell and explain how your solutions work. You have tried to make use of appropriate techniques and functions from the standard prelude to make your code elegant, although there are a few places where you could improve its elegance further. When prepending values to lists, avoid using <code>(++)</code> and instead favour <code>(:)</code> . You could also make further use of <code>let</code> bindings and <code>where</code> clauses to break down long expressions in your code. You could also make further use of suitable abstractions and implicitly recursive functions, e.g. <code>folds</code> , from the standard prelude instead of relying upon explicit recursion.
-----------------	--

Total mark:	68 / 100
-------------	----------